

AmericanGroupLLC Backend — Developer Integration Guide

This document is the definitive, 100% complete reference for developers building iOS, Android, and Web applications against the AmericanGroupLLC backend. It contains every API endpoint, all production credentials, exact environment variables, and copy-paste code for authentication flows.

If you are an app developer, everything you need to connect your app to the server, authenticate users, and fetch/save data is in this document.

1. Production Credentials & Environment

Your frontend application (whether React Native, Expo, Flutter, Next.js, or Vue) needs these exact environment variables to connect to the backend and third-party services.

Frontend .env Template (Copy & Paste)

```
# — API Gateway (The ONLY URL your app should ever call) —
EXP0_PUBLIC_API_URL=https://api.americangroupllc.com/api/v1

# — Supabase (For direct DB access if needed, though API Gateway is preferred) —
EXP0_PUBLIC_SUPABASE_URL=https://smvvjivvlprjhzhhoizym.supabase.co
EXP0_PUBLIC_SUPABASE_ANON_KEY=sb-publishable_nqtY6p48NKiRF53zivkpsQ_bRiqDSfc

# — Firebase (For Phone OTP / SMS Auth) —
EXP0_PUBLIC_FIREBASE_PROJECT_ID=americangroupllc-5bdfc
# (See section 2.1 for iOS/Android native Firebase config files)

# — Social Login Client IDs (For native SDKs) —
EXP0_PUBLIC_GOOGLE_CLIENT_ID=146431650883-blpfddrf32ureu4ucqlp3oku9jo07luq.apps.googleusercontent.com
EXP0_PUBLIC_FACEBOOK_APP_ID=1323978229267098
EXP0_PUBLIC_TWITTER_CLIENT_ID=vcL1rnmUGnAffCRIf3Czhdfjk
EXP0_PUBLIC_APPLE_CLIENT_ID=TLH7Z3G27A.com.americangroupllc.app
```

2. Platform Setup Guides

2.1. Mobile Apps (iOS & Android)

Mobile apps communicate directly with the API Gateway over HTTPS. They bypass CORS entirely.

Firebase Setup (Required for Phone OTP): You must download the native Firebase configuration files from the Firebase Console (Project: `americangroupllc-5bdfc`) and place them in your app:

- **Android:** Place `google-services.json` in `android/app/`
- **iOS:** Place `GoogleService-Info.plist` in `ios/Runner/` (Flutter) or `ios/YourApp/` (React Native)

Token Storage: Never store JWT tokens in `AsyncStorage` or `SharedPreferences` in plain text. You MUST use secure enclaves:

- **React Native / Expo:** Use `expo-secure-store`
- **Flutter:** Use `flutter_secure_storage`
- **Swift/Kotlin:** Use iOS Keychain / Android Keystore

2.2. Web Apps (React, Next.js, Vue)

Web apps are subject to strict CORS policies enforced by the API Gateway.

Allowed Origins: The backend currently accepts requests ONLY from:

- `https://app.americangroupllc.com`
- `https://americangroupllc.com`

- `http://localhost:3000`
- `http://localhost:8081`
- `capacitor://localhost`

Token Storage: Store the `access_token` in memory (e.g., React Context/Zustand) and the `refresh_token` in an `HttpOnly` cookie. Do not use `localStorage` for tokens to prevent XSS attacks.

3. Authentication & SSO Flows

The backend acts as a complete Identity Provider (IdP) handling OAuth 2.0, Phone OTP, and standard logins.

3.1. Social Logins (OAuth 2.0)

To implement “Sign in with Google/Apple/Facebook”, you do not need to handle the OAuth flow manually. Simply redirect the user to the backend’s start URL.

Provider	Redirect User To
Google	<code>https://api.americangroupllc.com/api/v1/auth/auth/social/google/start</code>
Apple	<code>https://api.americangroupllc.com/api/v1/auth/auth/social/apple/start</code>
Facebook	<code>https://api.americangroupllc.com/api/v1/auth/auth/social/facebook/start</code>
Twitter/X	<code>https://api.americangroupllc.com/api/v1/auth/auth/social/twitter/start</code>
GitHub	<code>https://api.americangroupllc.com/api/v1/auth/auth/social/github/start</code>
Microsoft	<code>https://api.americangroupllc.com/api/v1/auth/auth/social/microsoft/start</code>

How it works:

1. Your app opens a web browser/web view to the Start URL above.
2. The user logs in on the provider’s screen.
3. The backend catches the callback, creates/finds the user, and redirects back to your app with the tokens in the URL or via deep link (e.g., `yourapp://login?access_token=...`).

3.2. Phone OTP (Firebase SMS)

This is the standard flow for mobile apps using SMS verification.

Step 1: Request SMS (Client-side via Firebase SDK)

```
import auth from '@react-native-firebase/auth';

// Firebase sends the SMS directly to the user
const confirmation = await auth().signInWithPhoneNumber('+15551234567');
```

Step 2: Verify Code (Client-side)

```
// User types '123456'
const credential = await confirmation.confirm('123456');
const firebaseIdToken = await credential.user.getIdToken();
```

Step 3: Exchange Firebase Token for Backend JWT

```
const response = await fetch('https://api.americangroupllc.com/api/v1/auth/auth/firebase/verify', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ id_token: firebaseIdToken })
});

// Store these securely!
const { access_token, refresh_token, user_id } = await response.json();
```

(Note: For SIM-less testing in the simulator, you can use the test phone number configured in the Firebase Console and the test token:
 Ae0iMNCfLbdaWGFTYtXcHH_2YCDEBQTa1HMB4t0X-
 IWixXAHuQuX7qF984TzuLq70d0D40hEhKab6CsGyxHZe9tRxUTZlKVTS15PZrk1htEsnCI62SQkfvmHQ6Tp484IP2j6zoCnsxNKsH_QB12hau

3.3. Standard Email / Password

Register: POST /api/v1/auth/auth/register

```
{
  "email": "user@example.com",
  "password": "securepassword123"
}
```

Login: POST /api/v1/auth/auth/login

```
{
  "email": "user@example.com",
  "password": "securepassword123"
}
```

3.4. Token Management

All protected API routes require the access token in the header: Authorization: Bearer <access_token>

Refresh Token (when access_token expires): POST /api/v1/auth/auth/refresh

```
{
  "refresh_token": "<your_refresh_token>"
}
```

Get Current User Profile: GET /api/v1/auth/me (Requires Authorization header)

Logout: POST /api/v1/auth/auth/logout (Requires Authorization header)

4. Microservices API Reference

The backend is split into 10 domain services. All requests go through the API Gateway base URL:
 https://api.americangroupllc.com/api/v1 .

All endpoints below require Authorization: Bearer <access_token> unless marked (Public).

4.1. Travel Service (/travel)

- GET /health (Public)
- GET /flights?from=JFK&to=LHR&date=2026-08-15
- GET /hotels?city=London&checkIn=2026-08-15&guests=2

- GET /cars?city=London&date=2026-08-15
- POST /bookings
 - Body: { "type": "flight", "metadata": { "flight_id": "fl_101" } }
- GET /bookings

4.2. Retail Service (/retail)

- GET /health (Public)
- GET /products?q=laptop&category=electronics
- GET /grocery/deals?zipCode=90210
- POST /orders
 - Body: { "product_id": "prod_123", "quantity": 1, "total_amount": 1299.99 }
- GET /orders

4.3. Health Service (/health)

- GET /health (Public)
- GET /clinics?lat=34.05&lng=-118.24&radius=10
- POST /symptoms/analyze
 - Body: { "symptoms": ["headache", "fever"] }
- POST /appointments
 - Body: { "clinic_id": "cl_1", "date": "2026-07-01T10:00:00Z", "reason": "Checkup" }
- GET /appointments

4.4. Career Service (/career)

- GET /health (Public)
- GET /jobs?q=engineer&location=remote
- POST /resume/analyze
 - Body: { "resumeText": "Experienced software engineer...", "targetRole": "Senior Backend Developer" }
- POST /applications
 - Body: { "job_id": "job_99", "cover_letter": "I am interested..." }
- GET /applications

4.5. Real Estate Service (/realestate)

- GET /health (Public)
- GET /listings?city=Miami&minBeds=2&maxPrice=500000
- POST /properties/save
 - Body: { "property_id": "prop_88" }
- GET /properties/saved

4.6. Finance Service (/finance)

- GET /health (Public)
- POST /bills/analyze
 - Body: { "bills": [{ "provider": "PG&E", "amount": 150.50, "dueDate": "2026-07-05" }] }
- GET /mileage/deduction?miles=15000&year=2026
- POST /transactions
 - Body: { "type": "expense", "amount": 45.00, "description": "Uber ride" }

- GET /transactions

4.7. Community Service (/community)

- GET /health (Public)
- POST /alerts
 - Body: { "type": "traffic", "description": "Accident on I-5", "lat": 34.0, "lng": -118.2 }
- GET /alerts?lat=34.0&lng=-118.2&radius=5
- POST /posts
 - Body: { "content": "Hello neighborhood!", "group_id": "grp_1" }
- GET /posts?group_id=grp_1

4.8. AI Service (/ai)

- GET /health (Public)
- POST /chat
 - Body: { "message": "How do I save for a house?", "context": "finance" }
- GET /sessions
- POST /recommendations
 - Body: { "user_interests": ["travel", "tech"] }

4.9. Affiliate Service (/affiliate)

- GET /health (Public)
- GET /r/:token (Public) — Redirects user to affiliate destination
- POST /webhooks/postback/:partner (Public, HMAC secured) — For B2B partners
- GET /analytics

4.10. Ads Service (/ads)

(Internal routing only, not exposed directly to client apps)

5. Standard Error Handling

The API returns standard HTTP status codes. Every error response follows this exact JSON structure:

```
{
  "error": "Human-readable error description"
}
```

Common Codes:

- 200 / 201 : Success
 - 400 : Bad Request (missing parameters, invalid format)
 - 401 : Unauthorized (missing or expired Authorization header)
 - 403 : Forbidden (valid token, but lacks permission)
 - 404 : Not Found
 - 429 : Too Many Requests (Rate limit is 100 requests / minute / IP)
 - 500 : Internal Server Error
-

6. Developer Support

If you encounter issues connecting your application to the backend, require whitelist additions for CORS/OAuth callbacks, or need API Gateway logs reviewed, please contact:

Email: support@safecodeg.com